

Noise-Robust Voice Authentication Pipeline

A Multi-Modal Approach

Jakob Kauffmann

San José State University

May 6, 2025

Core Concept

- **Objective:** Fuse raw-waveform and spectrogram embeddings for speaker verification
- **Why:** Improve robustness to real-world noise using data augmentation (MUSAN dataset)
- **Approach:** Feature-level fusion via MLP classifier on dual embeddings

Data Augmentation

- **Noise Sources:** MUSAN corpus—babble, music, noise
- **SNR Levels:** 0, 5, 10, 15 dB applied per utterance
- **Filtering:**
 - Raw audio band-pass filtered (80–8000 Hz)
 - Gaussian smoothing before spectrogram computation

Raw-to-Spectrogram Pipeline

Code Snippet: Generating Mel Spectrograms

```
import torchaudio
from torchaudio.transforms import MelSpectrogram

def make_mel_spec(wav_path):
    sig, sr = torchaudio.load(wav_path)
    # Apply Gaussian filter
    sig = gaussian_filter(sig, sigma=1.0)
    # Compute mel-spectrogram
    mel_spec = MelSpectrogram(
        sample_rate=sr, n_mels=128,
        n_fft=1024, hop_length=256
    )(sig)
    return mel_spec.log2()
```

Custom SincNet Embedding

```
class SincNetEmbedding(nn.Module):
    def __init__(self):
        super().__init__()
        self.sinc_conv = SincConv_fast(1, 80, 101)
        self.bn = nn.BatchNorm1d(80)
        self.conv = nn.Sequential(
            nn.Conv1d(80,64,3), nn.ReLU(), # ...
        )
    def forward(self,x):
        x = self.sinc_conv(x)
        x = self.bn(x)
        x = self.conv(x)
        x = x.mean(-1)
        return F.normalize(x,dim=1)
```

Custom MobileNetV2 Embedding

```
class MobileNetEmbedding(nn.Module):
    def __init__(self):
        super().__init__()
        self.mbv2_features = MobileNetV2(width_mult=1.0).features
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(1280, 256)
    def forward(self, spec): # spec: [B,1,128,T]
        x = self.mbv2_features(spec)
        x = self.pool(x).view(x.size(0), -1)
        x = self.fc(x)
        return F.normalize(x, dim=1)
```

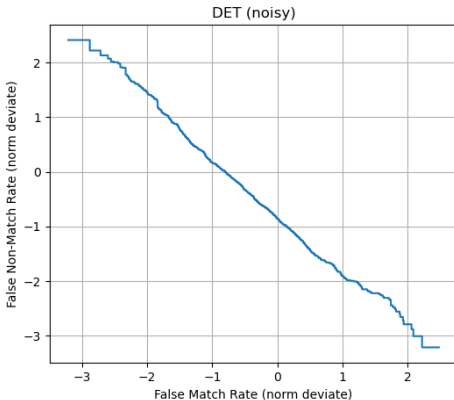
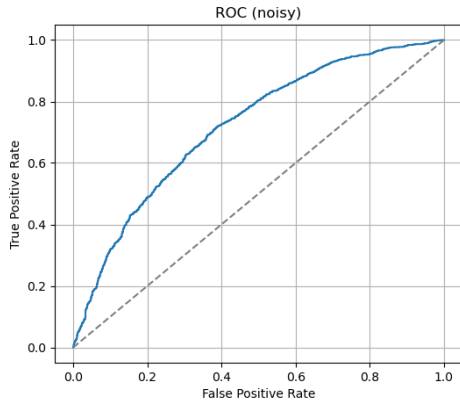
Fusion MLP Pipeline

```
class FusionMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(512,256)
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(256,1)
    def forward(self,emb_s,emb_m):
        x = torch.cat((emb_s,emb_m),1)
        x = F.relu(self.fc1(x))
        return torch.sigmoid(self.fc2(self.dropout(x)))
```

DeepFilterNet Experiment

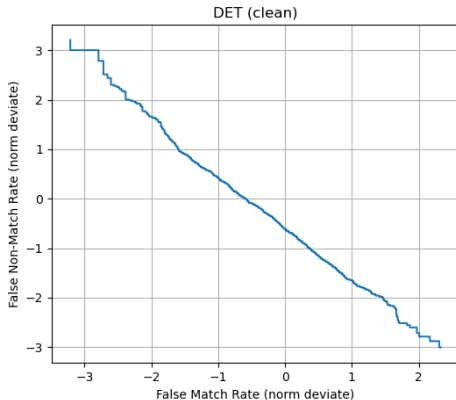
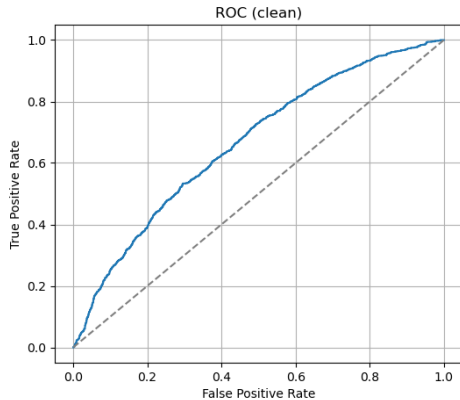
- Explored DeepFilterNet front-end for denoising
- Decided custom noisy-data training gave better generalization
- **Possible Next Step:** Mixed clean+noisy training or hybrid filter + augmentation

Results: Noisy Test Set



- EER: 34.1% @ -11.3260
- AUC: 0.72, d' : 0.82

Results: Clean Test Set



- EER: 38.6% @ -12.5464
- AUC: 0.66, d' : 0.60

Lessons Learned

- Custom data augmentation (MUSAN) critical for noise robustness
- Band-pass Gaussian filters improved feature quality
- Fusion architecture validated, but score overlap persists

Next Steps

- Scale to VoxCeleb2 + additional noise corpora
- Investigate ECAPA-TDNN + angular margin loss (ArcFace)
- Hybrid training: clean + noisy data mixes
- Combine DeepFilterNet pre-filtering with noisy augmentation
- Real-world deployment metrics: MTTE/MTTD measurement

Conclusions

- Demonstrated multi-modal, noise-robust pipeline
- Identified clear improvement avenues for future work in CS 297/298

Questions?